UNIVERSITY OF PETROLEUM AND ENERGY STUDIES

Dehradun, Uttarakhand

School of Computer Science

Containerization and DevOps

Project Assignment 1

Name: Daksh Mehrotra
SAP ID: 500125960
Course: BTech CSE
Specialization: CCVT
Batch: 2
Submitted To: Dr. Prateek Raj Gautam

Contents

1 Project Overview **2**

2 GitHub Repository **2**

3 System Architecture **2**

4 Dockerfiles for Individual Services **3**
4.1 Backend Dockerfile 3
4.2 Database Dockerfile 3

5 Docker Compose Configuration **4**

6 Custom IPVLAN Network Setup **5**

7 Verification and Testing **5**
7.1 Network Inspection 5
7.2 Container IP Verification 6

8 Volume Persistence Testing **9**

9 Build Optimization **10**

10 Image Size Comparison **11**

11 Macvlan vs IPVLAN **11**

12 Conclusion **11**

1 Project Overview

This project demonstrates the containerization and deployment of a three-tier web application using Docker and Docker Compose. The architecture consists of three independent services:

- Frontend – Responsible for the user interface.
- Backend – Handles API logic and communication with the database.
- Database – Stores persistent application data using PostgreSQL.

Each component runs inside its own container, ensuring isolation and scalability. The containers communicate through a custom IPVLAN network that assigns static IP addresses to each service.

The entire system is orchestrated using Docker Compose, which simplifies the process of building, running, and managing multi-container applications.

2 GitHub Repository

All source code and project configuration files are stored in a GitHub repository.

The repository includes:

- Backend source code
- Database initialization configuration
- Frontend UI code
- Dockerfiles for each service
- Docker Compose configuration
- Documentation and screenshots

GitHub Repository:
`https://github.com/DakshMehrotra/Containerization-and-DevOps/tree/main/Assignment-1`

3 System Architecture

The project follows a classic three-tier architecture.

$$\text{Client} \rightarrow \text{Frontend} \rightarrow \text{Backend} \rightarrow \text{Database}$$

- The **Frontend** provides the graphical interface for users.
- The **Backend API** processes requests and interacts with the database.
- The **Database** stores application data persistently.

Each component runs inside its own Docker container.

4 Dockerfiles for Individual Services

Each service in the application is built using its own Dockerfile.

4.1 Backend Dockerfile

Location:
`backend/Dockerfile`
The backend container runs a Node.js API server. The Dockerfile uses a multi-stage build to minimize image size and improve security.

```
FROM node:20-alpine AS builder

WORKDIR /app
COPY package.json .
RUN npm install --production
COPY server.js .

FROM node:20-alpine

WORKDIR /app
COPY --from=builder /app /app

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 5000
CMD ["node", "server.js"]
```

Key characteristics:

- Multi-stage build process
- Alpine Linux base image
- Non-root user execution
- Minimal runtime environment

4.2 Database Dockerfile

Location:
`database/Dockerfile`
The database service runs PostgreSQL and initializes the required database schema.
Environment variables configured:

- POSTGRES_DB
- POSTGRES_USER
- POSTGRES_PASSWORD

These variables allow flexible configuration of the database environment.

5 Docker Compose Configuration

Docker Compose is used to orchestrate all containers and ensure they start in the correct order.

The following services are defined:

- frontend
- backend
- database

```
version: "3.9"

services:

  database:
    build: ./database
    container_name: postgres_db
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      mynet:
        ipv4_address: 172.24.10.10

  backend:
    build: ./backend
    container_name: backend_api
    restart: always
    depends_on:
      - database
    environment:
      DB_HOST: 172.24.10.10
      DB_USER: appuser
      DB_PASSWORD: securepassword
      DB_NAME: appdb
    networks:
      mynet:
        ipv4_address: 172.24.10.20

  frontend:
    build: ./frontend
    container_name: frontend_ui
    restart: always
    networks:
      mynet:
        ipv4_address: 172.24.10.30

volumes:
  postgres_data:
```

```
networks:
  mynet:
    external: true
```

Important configuration features:

- Static IP assignment
- Persistent storage using volumes
- Automatic container restart
- Environment variable injection

6 Custom IPVLAN Network Setup

Before running Docker Compose, the custom network must be created manually.

```
docker network create -d ipvlan \
--subnet=172.24.0.0/16 \
--gateway=172.24.0.1 \
-o parent=eth0 \
mynet
```

This network allows containers to communicate directly using assigned IP addresses.

7 Verification and Testing

Several verification steps were performed to confirm the correct configuration.

7.1 Network Inspection

```
docker network inspect mynet
```

```
dakshmehrotra@LAPTOP-IQ7I38PQ:/mnt/d/Assignment1$ docker network inspect mynet
[
    {
        "Name": "mynet",
        "Id": "d47a8f034f476d88f1ab5faf039c3392d19cb56dcfb6f9a45d9eb91fa9b99779",
        "Created": "2026-03-16T12:47:40.738331816Z",
        "Scope": "local",
        "Driver": "ipvlan",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.24.0.0/16",
                    "Gateway": "172.24.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "477210c420bd20bcdc98db22af8c76990f2cbfc6661797f7730cc7fb5e9d5f06": {
                "Name": "frontend_ui",
                "EndpointID": "11368d8949b5f2741c1944be266a514306f7192a50328194104 6e1a23754783f",
                "MacAddress": "",
                "IPv4Address": "172.24.10.30/16",
                "IPv6Address": ""
            },
            "9086159b5f5d8dda218fb488ba3ffc2dc7537e6b4746918874c4f471c8bdcac2": {
                "Name": "postgres_db",
                "EndpointID": "cea817ea55d7e221fd8dbd96f9e221405789656db91139f1448db173d810524c",
                "MacAddress": "",
                "IPv4Address": "172.24.10.10/16",
                "IPv6Address": ""
            },
            "adce29760a5c7a1996e3c56ae084bc95683d36153eb89302d4d92cd83edcdb9c": {
                "Name": "backend_api",
                "EndpointID": "63b3ff6e23f3cc2f948139debb7a1b4e9aa215306de5ad409043df9155d4ce5b",
                "MacAddress": "",
                "IPv4Address": "172.24.10.20/16",
                "IPv6Address": ""
            }
        },
```

Figure 1: Docker network inspection output

7.2 Container IP Verification

Commands used:

```
docker inspect backend_api
docker inspect frontend_ui
docker inspect postgres_db
```

```
            "SandboxKey": "/var/run/docker/netns/3d6a77037785",
            "Ports": {},
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "",
            "Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "",
            "IPPrefixLen": 0,
            "IPv6Gateway": "",
            "MacAddress": "",
            "Networks": {
                "mynet": {
                    "IPAMConfig": {
                        "IPv4Address": "172.24.10.20"
                    },
                    "Links": null,
                    "Aliases": [
                        "backend",
                        "adce29760a5c"
                    ],
                    "MacAddress": "",
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "d47a8f034f476d88f1ab5faf039c3392d19cb56dcfb6f9a45d9eb91fa9b99779",
                    "EndpointID": "63b3ff6e23f3cc2f948139debb7a1b4e9aa215306de5ad409043df9155d4ce5b",
                    "Gateway": "172.24.0.1",
                    "IPAddress": "172.24.10.20",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": [
                        "backend_api",
                        "backend",
                        "adce29760a5c"
                    ]
                }
            }
        }
    }
]
```


Figure 2: Backend container inspection

```
            "SandboxID": "943552e9b999e96080a8c76e1ce999391015a62006b8dda0095af8875bd85584",
            "SandboxKey": "/var/run/docker/netns/943552e9b999",
            "Ports": {},
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "",
            "Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "",
            "IPPrefixLen": 0,
            "IPv6Gateway": "",
            "MacAddress": "",
            "Networks": {
                "mynet": {
                    "IPAMConfig": {
                        "IPv4Address": "172.24.10.30"
                    },
                    "Links": null,
                    "Aliases": [
                        "477210c420bd",
                        "frontend"
                    ],
                    "MacAddress": "",
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "d47a8f034f476d88f1ab5faf039c3392d19cb56dcfb6f9a45d9eb91fa9b99779",
                    "EndpointID": "11368d8949b5f2741c1944be266a514306f7192a503281941046e1a23754783f",
                    "Gateway": "172.24.0.1",
                    "IPAddress": "172.24.10.30",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": [
                        "frontend_ui",
                        "477210c420bd",
                        "frontend"
                    ]
                }
            }
        }
    }
```


Figure 3: Frontend container inspection

```
            "SandboxKey": "/var/run/docker/netns/016686b4026a",
            "Ports": {},
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "",
            "Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "",
            "IPPrefixLen": 0,
            "IPv6Gateway": "",
            "MacAddress": "",
            "Networks": {
                "mynet": {
                    "IPAMConfig": {
                        "IPv4Address": "172.24.10.10"
                    },
                    "Links": null,
                    "Aliases": [
                        "9086159b5f5d",
                        "database"
                    ],
                    "MacAddress": "",
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "d47a8f034f476d88f1ab5faf039c3392d19cb56dcfb6f9a45d9eb91fa9b99779",
                    "EndpointID": "cea817ea55d7e221fd8dbd96f9e221405789656db91139f1448db173d810524c",
                    "Gateway": "172.24.0.1",
                    "IPAddress": "172.24.10.10",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": [
                        "postgres_db",
                        "9086159b5f5d",
                        "database"
                    ]
                }
            }
        }
    }
]
```


Figure 4: Database container inspection

8 Volume Persistence Testing

To verify data persistence:

1. Data was inserted into the PostgreSQL database.
2. Containers were stopped using:

```
docker compose down
docker compose up -d
```

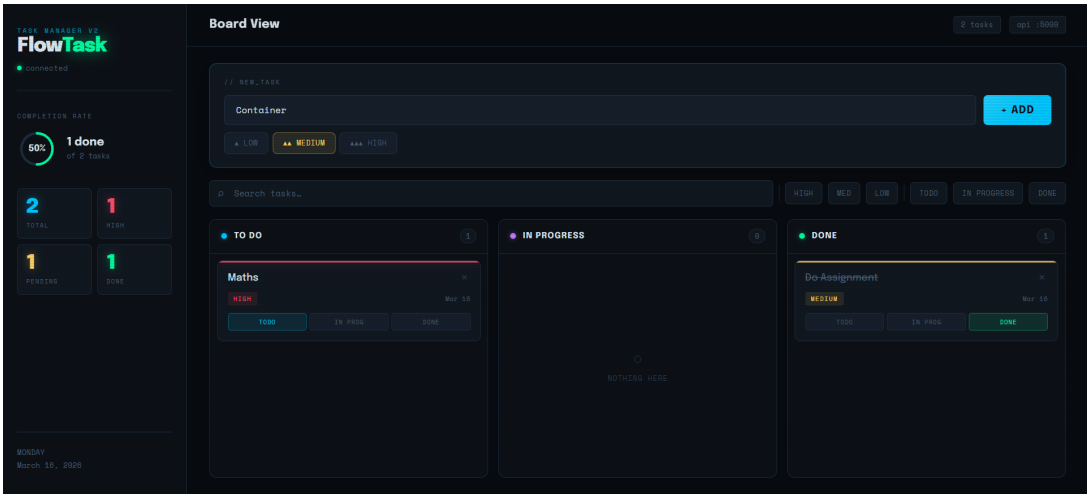

Figure 5: Database before container shutdown

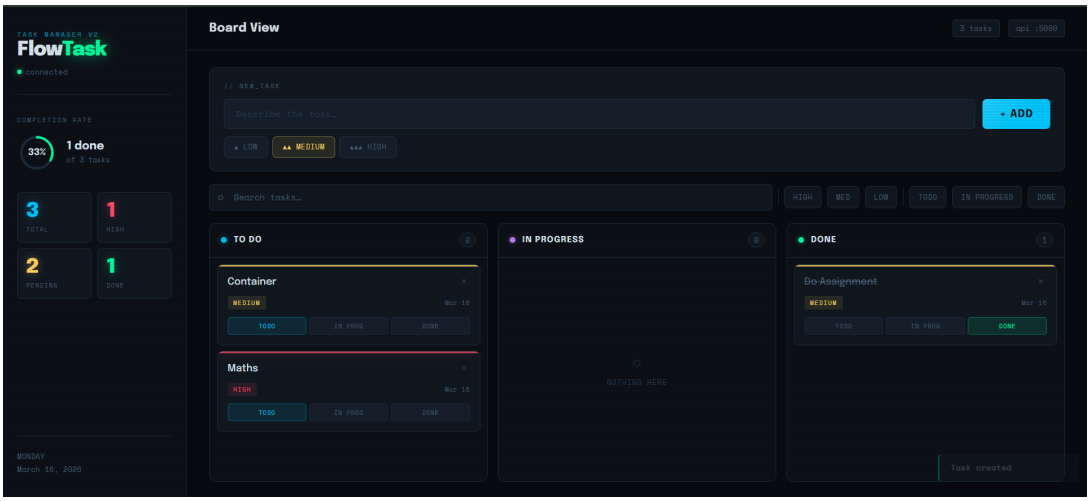

Figure 6: Database after container restart

The data remained intact after restarting the containers, confirming that the Docker volume was functioning correctly.

9 Build Optimization

The backend container image was optimized using several techniques:

- Multi-stage Docker builds
- Alpine Linux base image
- Non-root execution
- Reduced dependency footprint

These techniques significantly reduce image size and improve security.

10 Image Size Comparison

Two versions of the backend image were built:

- Standard image (single-stage build)
- Optimized image (multi-stage Alpine build)

```
dakshmehrotra@LAPTOP-IQ7I38PQ:/mnt/d/Assignment1$ docker images
REPOSITORY            TAG       IMAGE ID       CREATED          SIZE
<none>                <none>    38b9a8600262   8 seconds ago    1.1GB
assignment1_frontend  latest    851695093d5a   28 minutes ago   62.2MB
assignment1_backend   latest    c11ed163566c   28 minutes ago   138MB
```

Figure 7: Docker image size comparison

The optimized image is considerably smaller than the standard build.

11 Macvlan vs IPVLAN

Feature	Macvlan	IPVLAN
MAC Address	Unique MAC per container	Shared host MAC
Host Communication	Not supported by default	Supported
Isolation	Higher	Moderate
Use Case	Physical network simulation	Cloud / virtualization

IPVLAN was chosen for this project because it works reliably in virtualized environments such as WSL2 and VirtualBox where Macvlan often encounters networking limitations.

12 Conclusion

This project successfully demonstrates the containerization and deployment of a three-tier web application using Docker technologies. By using Docker Compose, container orchestration becomes simple and reproducible.

Key achievements include:

- Containerization of frontend, backend, and database
- Static IP networking using IPVLAN
- Persistent storage using Docker volumes
- Optimized container image builds
- Automated multi-container orchestration using Docker Compose

The approach used in this project reflects modern DevOps practices and provides a scalable and portable deployment model for cloud-native applications.